PRAKTIKUM 4
APPLICATION PROGRAMMING INTERFACE (API)

Dosen: Ir. Alam Rahmatulloh, S.T., M.T., MCE., IPM.

A. Tujuan Praktikum

1. Mahasiswa memahami konsep dasar REST API dan format data JSON.
2. Mahasiswa mampu mengimplementasikan pemrograman berorientasi objek (OOP) pada PHP.
3. Mahasiswa mampu memisahkan logika database (Model) dengan logika penyajian data (Endpoint).
4. Mahasiswa mampu melakukan pengujian API menggunakan alat bantu (Postman/Thunder Client).

B. Persiapan Lingkungan Kerja

Pastikan perangkat lunak berikut sudah berjalan:

1. **Web Server & Database:** Laragon V6.0.0 (download disini).
2. **Text Editor:** Visual Studio Code atau sejenisnya.
3. **API Client:** Postman atau Ekstensi VS Code "Thunder Client".
4. **Browser:** Google Chrome / Mozilla Firefox.Langkah Kerja Praktikum

C. Langkah Praktikum

Langkah 1: Membuat Class Database

Pada praktikum sebelumnya, koneksi dilakukan secara prosedural. Sekarang kita akan membungkusnya menjadi sebuah Class. Buat file Database.php di dalam folder config/.

```
<?php

class Database {
    private $host = "localhost";
    private $db_name = "kampus";
    private $username = "root";
    private $password = "";
    public $conn;

    public function getConnection() {
        $this->conn = null;
        try {
            $this->conn = new PDO("mysql:host=" . $this->host . ";dbname=" .
$this->db_name, $this->username, $this->password);
            $this->conn->setAttribute(PDO::ATTR_ERRMODE,
PDO::ERRMODE_EXCEPTION);
        } catch(PDOException $exception) {
```

```
        echo "Koneksi Gagal: " . $exception->getMessage();
    }
    return $this->conn;
  }
}

?>
```

Langkah 2: Membuat Class Model Mahasiswa

Buat file Mahasiswa.php di dalam folder models/. File ini berfungsi menangani logika query ke tabel mahasiswa.

1. Buat model awal mahasiswa.

```
<?php
class Mahasiswa {
   private $conn;
   private $table_name = "mahasiswa";

   public $id;
   public $nama;
   public $npm;
   public $jurusan;

   public function __construct($db) {
      $this->conn = $db;
   }
}
?>
```

2. Tambahkan method read() untuk mengambil data ke class mahasiswa.

```
public function read() {
   $query = "SELECT * FROM " . $this->table_name . " ORDER BY id DESC";
     $stmt = $this->conn->prepare($query);
     $stmt->execute();
     return $stmt;
}
```

3. Tambahkan method create() untuk menambah data ke class mahasiswa.

```
public function create() {
   $query = "INSERT INTO " . $this->table_name . " (npm, nama, jurusan)
VALUES (?, ?, ?)"; $stmt = $this->conn->prepare($query);
   if ($stmt->execute([$this->npm, $this->nama, $this->jurusan])) {
      return true;
```

```
    }

    return false;
}
```

4. Tambahkan method update pada class mahasiswa untuk mengupdate data.

```
public function update() {
    $query = "UPDATE " . $this->table_name . " SET npm = ?, nama = ?,
jurusan = ? WHERE id = ?"; $stmt = $this->conn->prepare($query);
    if ($stmt->execute([$this->npm, $this->nama, $this->jurusan, $this->id])) {
        return true;
    }
    return false;
}
```

5. Tambahkan method delete pada class mahasiswa untuk menghapus data.

```
public function delete() {
    $query = "DELETE FROM " . $this->table_name . " WHERE id = ?";
    $stmt = $this->conn->prepare($query);
    if ($stmt->execute([$this->id])) {
        return true;
    }
    return false;
}
```

Langkah 3: Membuat Endpoint API (READ)

Buat file read.php di dalam folder api/. File ini akan mengembalikan **JSON** data mahasiswa.

```
<?php

header("Access-Control-Allow-Origin: *");
header("Content-Type: application/json; charset=UTF-8");
header("Access-Control-Allow-Methods: GET");

if ($_SERVER['REQUEST_METHOD'] !== 'GET') {
    http_response_code(405);
    echo json_encode(["message" => "Method tidak diizinkan."]);
    exit;
}

include_once '../config/Database.php';
include_once '../models/Mahasiswa.php';
```

```
$database = new Database();
$db = $database->getConnection();

$mahasiswa = new Mahasiswa($db);
$stmt = $mahasiswa->read();
$num = $stmt->rowCount();

if($num > 0) {
    $mhs_arr = array();
    while ($row = $stmt->fetch(PDO::FETCH_ASSOC)) {
        extract($row);
        $mhs_item = array(
            "id" => $id,
            "nama" => $nama,
            "npm" => $npm,
            "jurusan" => $jurusan
        );
        array_push($mhs_arr, $mhs_item);
    }
    http_response_code(200);
    echo json_encode($mhs_arr);
} else {
    http_response_code(404);
    echo json_encode(array("message" => "Data tidak ditemukan."));
}
?>
```

Langkah 4: Membuat Endpoint API (CREATE)

Buat file create.php di dalam folder api/. File ini akan memproses request penambahan data mahasiswa.

```
<?php

header("Access-Control-Allow-Origin: *");
header("Content-Type: application/json; charset=UTF-8");
header("Access-Control-Allow-Methods: POST");

if ($_SERVER['REQUEST_METHOD'] !== 'POST') {
    http_response_code(405);
    echo json_encode(["message" => "Method tidak diizinkan."]);
    exit;
}
```

```
include_once '../config/Database.php';
include_once '../models/Mahasiswa.php';

$database = new Database();
$db = $database->getConnection();
$mahasiswa = new Mahasiswa($db);

$data = json_decode(file_get_contents("php://input"));

if(!empty($data->npm) && !empty($data->nama) && !empty($data->jurusan)) {
    $mahasiswa->npm = $data->npm;
    $mahasiswa->nama = $data->nama;
    $mahasiswa->jurusan = $data->jurusan;

    if($mahasiswa->create()) {
        http_response_code(201); // Created
        echo json_encode(array("message" => "Mahasiswa berhasil
ditambahkan."));
    } else {
        http_response_code(503); // Service Unavailable
        echo json_encode(array("message" => "Gagal menambahkan
mahasiswa."));
    }
} else {
    http_response_code(400); // Bad Request
    echo json_encode(array("message" => "Data tidak lengkap."));
}
?>
```

Langkah 5: Membuat Endpoint API (UPDATE)

Buat file update.php di dalam folder api/. File ini akan memproses request pengubahan data mahasiswa.

```
<?php

header("Access-Control-Allow-Origin: *");
header("Content-Type: application/json; charset=UTF-8");
header("Access-Control-Allow-Methods: PUT");

if ($_SERVER['REQUEST_METHOD'] !== 'PUT') {
    http_response_code(405);
    echo json_encode(["message" => "Method tidak diizinkan."]);
    exit;
```

```
}

include_once '../config/Database.php';
include_once '../models/Mahasiswa.php';

$database = new Database();
$db = $database->getConnection();
$mahasiswa = new Mahasiswa($db);

$data = json_decode(file_get_contents("php://input"));

$mahasiswa->id = $data->id;
$mahasiswa->npm = $data->npm;
$mahasiswa->nama = $data->nama;
$mahasiswa->jurusan = $data->jurusan;

if($mahasiswa->update()) {
    http_response_code(200);
    echo json_encode(array("message" => "Data mahasiswa berhasil
diperbarui."));
} else {
    http_response_code(503);
    echo json_encode(array("message" => "Gagal memperbarui data."));
}
?>
```

Langkah 6: Membuat Endpoint API (DELETE)

Buat file delete.php di dalam folder api/. File ini akan memproses request hapus data mahasiswa.

```
<?php

header("Access-Control-Allow-Origin: *");
header("Content-Type: application/json; charset=UTF-8");
header("Access-Control-Allow-Methods: DELETE");

if ($_SERVER['REQUEST_METHOD'] !== 'DELETE') {
    http_response_code(405);
    echo json_encode(["message" => "Method tidak diizinkan."]);
    exit;
}

include_once '../config/Database.php';
```

```
include_once '../models/Mahasiswa.php';

$database = new Database();
$db = $database->getConnection();
$mahasiswa = new Mahasiswa($db);

$data = json_decode(file_get_contents("php://input"));

$mahasiswa->id = $data->id;

if($mahasiswa->delete()) {
    http_response_code(200);
    echo json_encode(array("message" => "Mahasiswa berhasil dihapus."));
} else {
    http_response_code(503);
    echo json_encode(array("message" => "Gagal menghapus mahasiswa."));
}
?>
```

Langkah 7: Pengujian

Lakukan pengujian menggunakan API client yang anda gunakan pada contoh dengan postman.

1. Buka Postman
2. Pilih method GET
3. Masukkan URL http://localhost/api/read.php
4. Klik send dan lihat hasilnya pada bagian response
5. Lakukan pengujian pada method lain.

Langkah 8: Upload Kode ke Github

Buat repository github, lalu push kode ke repository github seperti pada praktikum 1.

Langkah 9: Siapkan Database untuk Aplikasi Server

Masuk ke server dan buat user juga database sesuai dengan kebutuhan.

1. Masuk ke mysql dengan commmand

```
Mysql -u root
```

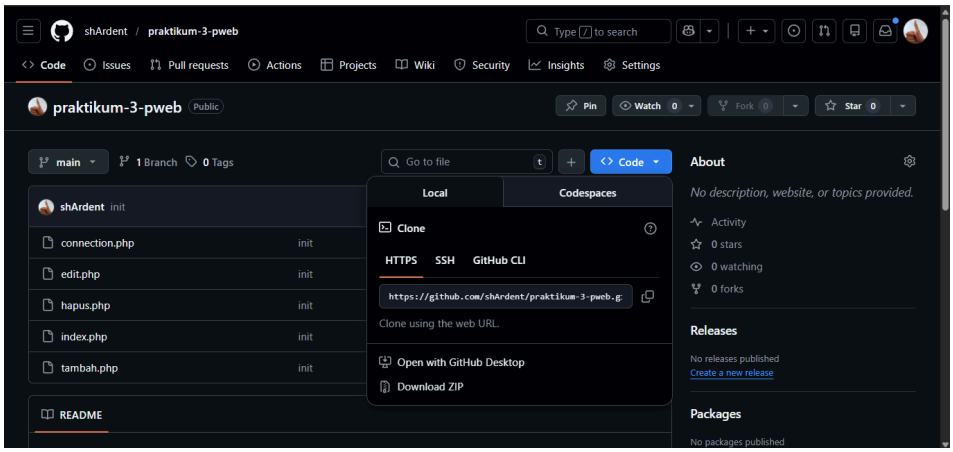
2. Buat database seperti pada laptop local anda (ulangi langkah 1.4 dan 1.5).
3. Buat user baru dan berikan kases ke database yang sudah dibuat.

```
CREATE USER CREATE USER 'user'@'localhost' IDENTIFIED BY 'password';
GRANT ALL PRIVILEGES ON kampus.* TO 'user'@'localhost';
FLUSH PRIVILEGES;
```

Langkah 10: Clone Kode dari Github

Clone script yang sudah diupload di github ke server.

1. Copy url repository yang sudah dibuat.



2. Masuk ke server dan buat direktori bernama praktikum di "/var/www/" (gunakan user root).

```
cd /var/www/

mkdir praktikum

cd praktikum
```

3. Clone repositori github ke direktori praktikum.

```
## ganti url dengan url repositori github anda

git clone https://github.com/shArdent/praktikum-3-pweb.git .
```

4. Ganti kredensial koneksi database dengan kredensial baru yang anda buat di server (sesuaikan "connection.php" dengan nama file yang anda buat sendiri).

```
nano config/Database.php
```

5. Simpan perubahan dengan ctrl+s lalu ctrl+x.

Langkah 11: Ubah konfigurasi apache

Perbarui konfigurasi apache agar tidak menggunakan config default.

1. Masuk ke direktori config apache.

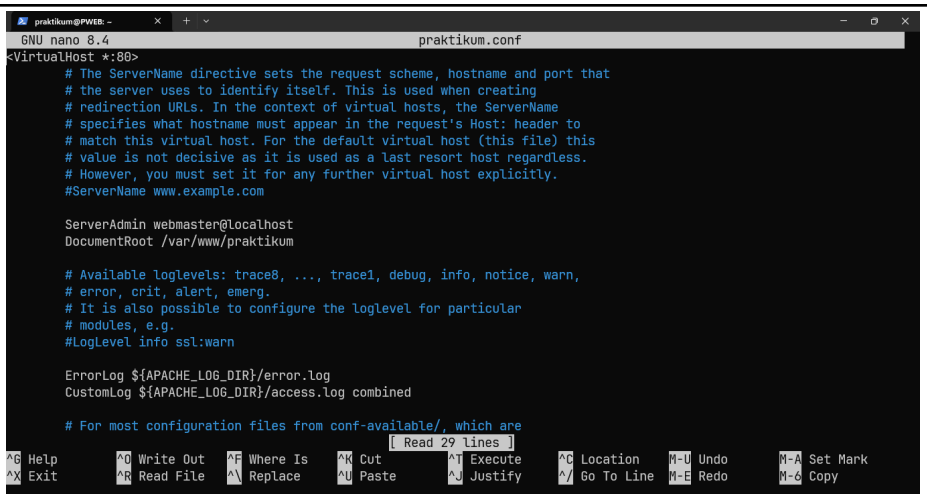
```
cd /etc/apache2/sites-available
```

2. Copy config default dengan nama praktikum.conf

```
cp 000-default.conf praktikum.conf
```

3. Ubah baris DocumentRoot menjadi direktori tempat script anda berada.

```
nano praktikum.conf
```



4. Simpan perubahan dengan ctrl+s dan ctrl+x.

5. Matikan config default apache.

```
a2dissite 000-default.conf
```

6. Aktifkan config baru praktikum.conf.

```
a2ensite praktikum.conf
```

7. Reload apache.

```
systemctl reload apache2
```

8. Buka browser dan masukan ip server anda. Pastikan halaman yang anda buat tampil dan berfungsi.